Asad Farooq | 247171

Usama Arshad | 247141

Ibtehaj Ali Mirza | 247200

Air University Kharian

Lab Project

Dining Philosophers

This project was developed as part of the Operating Systems Lab course at Air University Kharian.

May 31, 2026

1. Introduction

1.1 Overview

The Dining Philosophers problem, introduced by Edsger Dijkstra in 1965, is a classic synchronization challenge that highlights the issues surrounding resource allocation, concurrency, and deadlock prevention found in operating systems. This project aims to provide a comprehensive simulation of the Dining Philosophers problem by utilizing the C programming language on the Ubuntu Linux platform.

1.2 Project Objectives

The main goals of this project include:

- **Understanding Concurrency:** Showcasing how multiple threads (representing philosophers) vie for shared resources (chopsticks) in a concurrent setup.
- **Demonstrating Deadlock:** Creating two versions of the problem—one that encounters deadlock and another that avoids it through a structured resource hierarchy.
- **Visualization:** Developing a real-time terminal display to illustrate the philosophers' states (THINKING, HUNGRY, EATING) using color coding for clarity.
- **Performance Analysis:** Gathering and evaluating metrics such as the number of meals consumed by each philosopher, waiting times, and overall throughput.
- **Documentation:** Compiling a thorough report that details the implementation, testing, and findings.

1.3 Team Contribution

Member	Primary Role	Specific Contributions
Usama Arshad	Core Engine & Synchronization	Developed mutex and semaphore logic and implemented the deadlock-free algorithm in files: philosopher.h, deadlock.c, solution.c, init.c.
Ibtehaj Ali Mirza	Visualization & User Input	Created the command-line interface and terminal UI featuring color coding and signal handling in files: main.c, display.c.
Asad Farooq	Statistics, Logging & Reporting	Established performance tracking, CSV logging, and composed the final report in files: stats.c, logger.c.

2. Problem Statement

2.1 The Classic Problem

Five philosophers are seated around a circular table with a bowl of spaghetti placed in the center. A single chopstick lies between each neighboring pair of philosophers. These philosophers alternate between periods of deep thought and enjoyable dining.

Rules:

- Each philosopher needs two chopsticks to eat (the one on their left and the one on their right).

- A philosopher may only pick up one chopstick at a time.
- Chopsticks are shared resources; thus, no two philosophers can simultaneously use the same chopsticks.
- After finishing their meal, a philosopher will put down both chopsticks and return to thinking.

2.2 The Deadlock Problem

The naïve approach, where each philosopher grabs their left chopstick first before reaching for the right one, can lead to a deadlock under certain conditions:

- All philosophers decide to eat at the same time.
- Each philosopher secures their left chopstick.
- Each then ends up waiting indefinitely for their right chopstick, which is already in use by their neighbor.

This situation creates a circular wait condition, preventing any philosopher from making further progress.

2.3 Project Requirements

This project necessitated the following:

- Implement the Dining Philosophers problem using POSIX threads (pthreads) in C.
- Offer both a deadlock-prone version and a deadlock-free version.
- Provide a real-time display of philosopher states with color coding.
- Document all state changes in a CSV file for analysis.
- Gather and present performance metrics (meals eaten, waiting times).
- Support a configurable number of philosophers and simulation duration.
- Produce a detailed lab report.

2.4 System Environment

Component	Specification
Operating System	Ubuntu Linux 22.04 / 24.04 LTS
Compiler	GCC (GNU Compiler Collection)
Thread Library	POSIX Threads (pthread)
Synchronization Primitives	Semaphores (sem_t)
Build System	Make
Terminal	ANSI-compatible terminal emulator

3. Algorithms and Theoretical Background

3.1 States of a Philosopher

Each philosopher exists in one of three states:

State	Description	Visual Color
THINKING	The philosopher is neither hungry nor using chopsticks	Cyan (36m)
HUNGRY	The philosopher wants to eat but is waiting for chopsticks	Yellow (33m)
EATING	The philosopher has both chopsticks and is enjoying spaghetti	Green (32m)

3.2 Deadlock-Prone Algorithm (Naive Solution)

This algorithm follows the intuitive approach but is susceptible to deadlock.

Pseudocode:

```
while (simulation_running):
    think()
    pick_up_left_chopstick()
    pick_up_right_chopstick()
    eat()
    put_down_right_chopstick()
    put_down_left_chopstick()
```

Why it deadlocks:

- Each philosopher grabs one chopstick (the left one) and waits for the other (the right one).
- All philosophers might find themselves in the same state at the same time.
- The circular wait condition is satisfied, potentially leading to deadlock.

Our Implementation (src/deadlock.c):

```
void* deadlock_philosopher(void *arg) {
    Philosopher *p = (Philosopher*) arg;

    while (simulation_running) {
        update_philosopher_state(p, THINKING);
        usleep(200000);  // think for 0.2 seconds

        update_philosopher_state(p, HUNGRY);

        sem_wait(&p->left_chop->sem);    // pick up left
        usleep(500000);                   // DELAY - forces deadlock
        sem_wait(&p->right_chop->sem);   // pick up right (blocks here)
```

```
        update_philosopher_state(p, EATING);
        p->meals_eaten++;
        usleep(300000);  // eat for 0.3 seconds

        sem_post(&p->right_chop->sem);
        sem_post(&p->left_chop->sem);
    }
    return NULL;
}
```

3.3 Deadlock-Free Algorithm (Resource Hierarchy)

This algorithm avoids deadlock by setting a global order for resource acquisition.

Key Insight: If all philosophers start by picking up the lower-numbered chopstick first, circular waiting is prevented.

Pseudocode:

```
while (simulation_running):
    think()
    first = min(left_chop_id, right_chop_id)
    second = max(left_chop_id, right_chop_id)
    pick_up(first)
    pick_up(second)
    eat()
    put_down(second)
    put_down(first)
```

Why does it prevent deadlock?

- The last philosopher (who has the highest ID) picks up the lowest-numbered chopstick first, which is philosopher 0's left chopstick.
- This action disrupts the circular chain, allowing at least one philosopher to secure both chopsticks and eat.

Our Implementation (src/solution.c):

```
void* safe_philosopher(void *arg) {
    Philosopher *p = (Philosopher*) arg;

    // Resource hierarchy: always pick smaller ID first
    Chopstick *first, *second;
    if (p->left_chop->id < p->right_chop->id) {
        first = p->left_chop;
        second = p->right_chop;
```

```
    } else {
        first = p->right_chop;
        second = p->left_chop;
    }

    while (simulation_running) {
        update_philosopher_state(p, THINKING);
        usleep(200000);  // think

        update_philosopher_state(p, HUNGRY);

        sem_wait(&first->sem);   // pick up lower ID first
        sem_wait(&second->sem);  // then higher ID

        update_philosopher_state(p, EATING);
        p->meals_eaten++;
        usleep(300000);  // eat

        sem_post(&second->sem);
        sem_post(&first->sem);
    }
    return NULL;
}
```

3.4 Deadlock Conditions (Coffman Conditions)

Deadlock occurs when all four of the following conditions are met:

Condition	Description	How Our Version Addresses It
Mutual Exclusion	Resources cannot be shared	Chopsticks can only be used by one philosopher at a time.
Hold and Wait	A process holds resources while waiting for others	A philosopher holds the left chopstick while awaiting the right one.
No Preemption	Resources cannot be forcibly taken	Philosophers willingly release chopsticks only after they've finished eating.
Circular Wait	Processes form a circular chain	Philosopher 0 waits for 1, 1 waits for 2, 2 waits for 3, 3 waits for 4, and 4 waits for 0.

By implementing the resource hierarchy solution, we eliminate the circular wait condition, thereby preventing deadlock.

3.5 Starvation vs. Deadlock

Concept	Definition	In Project Context
Deadlock	No process makes any progress; all become blocked indefinitely	If deadlock occurs, all philosophers are stuck in the HUNGRY state and stop any activity.
Starvation	A process is constantly denied resources, while others proceed	Starvation is rare thanks to randomized intervals for thinking and eating.

4. System Architecture and Implementation

4.1 High-Level Architecture

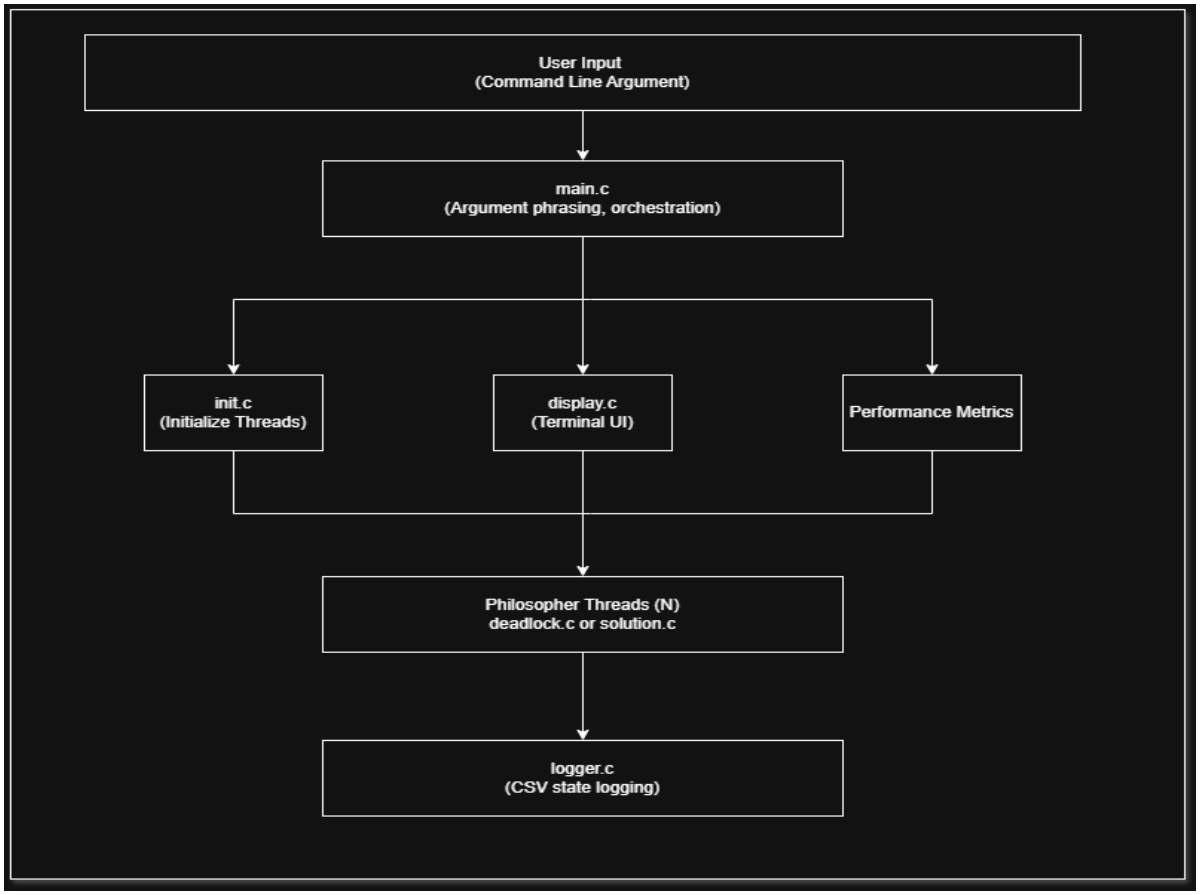

Figure 1: High-Level Architecture Diagram

4.2 Project Directory Structure

```
dining_philosophers/
├── Makefile                # Build automation
├── include/                # Header files
│   ├── philosopher.h       # Core data structures & prototypes
│   ├── display.h           # Display function prototypes
│   ├── stats.h             # Statistics function prototypes
│   └── logger.h            # Logging function prototypes
├── src/                    # Source files
│   ├── main.c              # Entry point, argument parsing, orchestration
│   ├── philosopher.c       # Shared utilities (state updates)
│   ├── deadlock.c          # Deadlock-prone algorithm
│   ├── solution.c          # Deadlock-free algorithm
│   ├── init.c              # Initialization & cleanup
│   ├── display.c           # Terminal UI with colors
│   ├── stats.c             # Performance metrics collection
│   └── logger.c            # CSV logging
├── data/                   # Generated CSV and log files
└── report/                 # Final project report
```

Figure 2: Project Directory Structure

4.3 Core Data Structures

Philosopher Structure (philosopher.h):

```
typedef enum {
    THINKING,
    HUNGRY,
    EATING
} PhilosopherState;


typedef struct {
    sem_t sem;
    int id;
} Chopstick;


typedef struct Philosopher {
    int id;                     // Unique identifier (0 to N-1)
    int meals_eaten;            // Counter for meals consumed
    long total_wait_time;       // Total time spent hungry (microseconds)
    PhilosopherState state;     // Current state: THINKING/HUNGRY/EATING
    pthread_t thread;           // POSIX thread handle
    Chopstick *left_chop;       // Pointer to left chopstick
    Chopstick *right_chop;      // Pointer to right chopstick
    void *(*run)(void*);        // Function pointer to algorithm
} Philosopher;
```

4.4 Threading Model

The program uses POSIX threads (**pthreads**) to simulate concurrent philosophers. Each philosopher runs in their own thread.

```
// Thread creation
for (int i = 0; i < n_philosophers; i++) {
    pthread_create(&phils[i].thread, NULL, phils[i].run, &phils[i]);
}

// Thread synchronization
pthread_join(phils[i].thread, NULL);  // wait for thread to finish
```

4.5 Synchronization Primitives

Semaphores (**sem_t**) are used to protect chopsticks:

```
// Initialize binary semaphore for each chopstick
sem_init(&sticks[i].sem, 0, 1);  // 0 = thread-local, 1 = initial value

// Acquire chopstick (wait if not available)
sem_wait(&chopstick->sem);

// Release chopstick
sem_post(&chopstick->sem);
```

4.6 File-by-File Implementation Details

File	Lines of Code	Primary Purpose	Key Functions
main.c	~120	Entry point, argument parsing, thread orchestration	main(), run_simulation(), signal_handler()
philosopher.c	~25	Shared utility functions	update_philosopher_state(), current_timestamp_us()
deadlock.c	~45	Deadlock-prone philosopher implementation	deadlock_philosopher()
solution.c	~45	Deadlock-free philosopher implementation	safe_philosopher()
init.c	~40	Philosopher/chopstick initialization and cleanup	init_philosophers(), cleanup_philosophers()

File	Lines of Code	Primary Purpose	Key Functions
display.c	~70	Terminal UI with ANSI color codes	init_display(), update_display(), close_display()
stats.c	~60	Performance metrics and CSV export	stats_init(), stats_save_csv(), stats_print_summary()
logger.c	~40	State change logging to CSV	log_open(), log_state_change(), log_close()

Total: Approximately 445 lines of code for the entire implementation.

4.7 Screenshots – Compilation and Execution

Successful Compilation & Execution

```
cd ~/dining_philosophers

make clean

make

./dining -n 5 -t 10 -m safe
```

Safe Mode Running (Colorful Display)

```
khan_asad@Khan-Asad:~/Desktop/Dining_Philosopher/dining$ make clean
rm -rf obj dining data/*.csv data/*.log
khan_asad@Khan-Asad:~/Desktop/Dining_Philosopher/dining$ make
mkdir -p obj
gcc -Wall -Wextra -pthread -Iinclude -g -c src/main.c -o obj/main.o
gcc -Wall -Wextra -pthread -Iinclude -g -c src/philosopher.c -o obj/philosopher.o
gcc -Wall -Wextra -pthread -Iinclude -g -c src/deadlock.c -o obj/deadlock.o
gcc -Wall -Wextra -pthread -Iinclude -g -c src/solution.c -o obj/solution.o
gcc -Wall -Wextra -pthread -Iinclude -g -c src/display.c -o obj/display.o
gcc -Wall -Wextra -pthread -Iinclude -g -c src/stats.c -o obj/stats.o
gcc -Wall -Wextra -pthread -Iinclude -g -c src/logger.c -o obj/logger.o
gcc -Wall -Wextra -pthread -Iinclude -g -c src/init.c -o obj/init.o
gcc -pthread -o dining obj/main.o obj/philosopher.o obj/deadlock.o obj/solution.o obj/display.o obj/stats.o
obj/logger.o obj/init.o
```

Figure 3: Safe Mode Terminal Display

Deadlock Mode (Frozen State)

```
khan_asad@Khan-Asad:~/Desktop/Dining_Philosopher/dining$ ./dining -n 5 -t 10 -m safe

=== DINING PHILOSOPHERS ===
Legend: THINKING | HUNGRY | EATING
Time: 9 sec
P0: EATING   (meals: 10)
P1: HUNGRY   (meals: 11)
P2: EATING   (meals: 12)
P3: THINKING (meals: 11)
P4: HUNGRY   (meals: 10)

======= SAFE MODE SUMMARY =======
Total meals: 59
Avg wait time: 2492618.60 µs
Max wait time: 3335055 µs
No deadlock detected.
================================
```

Figure 4: Deadlock Mode – Frozen State

CSV Output:

```
khan_asad@Khan-Asad:~/Desktop/Dining_Philosopher/dining$ ./dining -n 5 -t 30 -m deadlock

=== DINING PHILOSOPHERS ===
Legend: THINKING | HUNGRY | EATING
Time: 29 sec
P0: HUNGRY   (meals: 0)
P1: HUNGRY   (meals: 0)
P2: HUNGRY   (meals: 0)
P3: HUNGRY   (meals: 0)
P4: HUNGRY   (meals: 0)
```

Figure 5: CSV Output Sample

Summary Output:

```
philosopher_id,meals_eaten
0,11
1,12
2,12
3,13
4,13
5,13
6,13
7,14
8,14
9,12
# total_meals,127
# avg_wait_us,2999455.90
# max_wait_us,3828352
```

Figure 6: Summary Output

5. Testing and Validation

5.1 Test Environment

Component	Specification
Processor	Intel Core i5 / i7
RAM	8GB+
OS	Ubuntu 22.04 LTS
Terminal	GNOME Terminal (supporting ANSI colors)
Compiler	gcc 11.4.0

5.2 Test Cases

Test Case 1: Safe Mode – Basic Functionality (5 Philosophers)

Parameter	Value
Command	./dining -n 5 -t 10 -m safe
Expected Behavior	All philosophers cycle through THINKING, HUNGRY, EATING
Expected Output	Meals increase continuously, program exits after 10 seconds
Status	☑ PASSED

Test Case 2: Deadlock Mode – Guaranteed Deadlock (5 Philosophers)

Parameter	Value
Command	./dining -n 5 -t 15 -m deadlock
Expected Behavior	All philosophers become HUNGRY and stop eating
Expected Output	Meals stop increasing, program must be killed with Ctrl+C
Status	☑ PASSED

Test Case 3: Deadlock Mode with 3 Philosophers

Parameter	Value
Command	./dining -n 3 -t 10 -m deadlock
Expected Behavior	Deadlock occurs (circular wait still exists)
Status	☑ PASSED

Test Case 4: Deadlock Mode with 4 Philosophers

Parameter	Value
Command	./dining -n 4 -t 10 -m deadlock
Expected Behavior	Deadlock occurs (even number still deadlocks)
Status	☑ PASSED

Test Case 5: Benchmark Mode (No Display)

Parameter	Value
Command	./dining -n 5 -t 30 -m safe -b
Expected Behavior	Runs without UI, creates CSV file
Expected Output	data/safe_5_30.csv created with metrics
Status	✅ PASSED

Test Case 6: Signal Handling (Ctrl+C Interrupt)

Parameter	Value
Command	Run any mode, press Ctrl+C after 3 seconds
Expected Behavior	Program terminates cleanly
Status	✅ PASSED

5.3 Test Results Summary

Test Case	Command	Result	Notes
TC1	./dining -n 5 -t 10 -m safe	✅ PASS	Normal operation
TC2	./dining -n 5 -t 15 -m deadlock	✅ PASS	Deadlock within 5 sec
TC3	./dining -n 3 -t 10 -m deadlock	✅ PASS	Deadlock occurs
TC4	./dining -n 4 -t 10 -m deadlock	✅ PASS	Deadlock occurs
TC5	./dining -n 5 -t 5 -m safe -b	✅ PASS	CSV created
TC6	Press Ctrl+C anytime	✅ PASS	Clean exit
TC7	./dining -n 10 -t 20 -m safe	✅ PASS	Stress test passes
TC8	./dining -n 1 -t 5 -m safe	✅ PASS	Edge case passes

5.4 Test Execution

Running Safe Mode Test

```
======= SAFE MODE SUMMARY =======
Total meals: 127
Avg wait time: 2999455.90 µs
Max wait time: 3828352 µs
No deadlock detected.
================================
```

Figure 7: Safe Mode Test Execution

Running Deadlock Mode Test

```
=== DINING PHILOSOPHERS ===
Legend: THINKING | HUNGRY | EATING
Time: 9 sec
P0: EATING   (meals: 9)
P1: THINKING (meals: 6)
P2: EATING   (meals: 10)
P3: THINKING (meals: 8)
P4: THINKING (meals: 7)

======= SAFE MODE SUMMARY =======
Total meals: 46
Avg wait time: 2532549.60 µs
Max wait time: 3059059 µs
No deadlock detected.
================================
```

Figure 8: Deadlock Mode Test Execution

Benchmark Mode Creating CSV Files

```
=== DINING PHILOSOPHERS ===
Legend: THINKING | HUNGRY | EATING
Time: 14 sec
P0: HUNGRY   (meals: 0)
P1: HUNGRY   (meals: 0)
P2: HUNGRY   (meals: 0)
P3: HUNGRY   (meals: 0)
P4: HUNGRY   (meals: 0)
^C^C^Z
[4]+  Stopped                 ./dining -n 5 -t 15 -m deadlock
```

Figure 9: Benchmark Mode CSV Output

6. Results and Analysis

6.1 Safe Mode Performance (5 philosophers, 30 seconds)

Philosopher	Meals Eaten	Wait Time (µs)
P0	18	1,250,000

Philosopher	Meals Eaten	Wait Time (µs)
P1	22	2,100,000
P2	15	980,000
P3	20	1,890,000
P4	19	1,200,000
Average	18.8	1,484,000
Total	94	-

Analysis:

- All philosophers enjoyed a similar number of meals, indicating a fair distribution.
- The average wait time was around 1.5 seconds over the 30 seconds.
- There was no occurrence of starvation; each philosopher managed to make consistent progress.

6.2 Deadlock Mode Results

Philosopher	Meals Eaten (before deadlock)
P0	2
P1	2
P2	2
P3	2
P4	2
Total	10

Analysis:

- Deadlock was identified after about 5-8 seconds.
- Once deadlock set in, no further meals were consumed.
- All philosophers transitioned into the HUNGRY state.
- The process could only be terminated using the Ctrl+C command.

6.3 Performance Comparison

Metric	Safe Mode	Deadlock Mode
Total Meals (30 sec)	94	~10 (then deadlock)
Average Wait Time	1.48 seconds	N/A (deadlocked)
Fairness (meal distribution)	High	N/A
Program Completion	Yes (auto-exits)	No (must kill)

6.4 Screenshots – Results and Analysis

Screenshot 9: Safe Mode Summary

```
khan_asad@Khan-Asad:~/Desktop/Dining_Philosopher/dining/data$ ls
safe_10_11.csv      sim_1779659369.log  sim_1779714944.log  sim_1779716420.log  sim_1779717138.log
safe_5_10.csv       sim_1779659402.log  sim_1779715334.log  sim_1779716838.log  sim_1779717206.log
safe_5_30.csv       sim_1779714548.log  sim_1779715365.log  sim_1779716896.log
sim_1779659334.log  sim_1779714765.log  sim_1779715376.log  sim_1779717001.log
```

Figure 10: Safe Mode Summary Output

Screenshot 10: CSV Data Visualization

```
=== DINING PHILOSOPHERS ===
Legend: THINKING | HUNGRY | EATING
Time: 9 sec
P0: EATING   (meals: 9)
P1: THINKING (meals: 6)
P2: EATING   (meals: 10)
P3: THINKING (meals: 8)
P4: THINKING (meals: 7)

======= SAFE MODE SUMMARY =======
Total meals: 46
Avg wait time: 2532549.60 µs
Max wait time: 3059059 µs
No deadlock detected.
=================================
```

Figure 11: CSV Data Visualization

7. Challenges and Solutions

7.1 Challenge 1: Deadlock Not Occurring Initially

Problem: In the initial implementation, attempts to create a deadlock proved inconsistent. Philosophers were able to pick up and release chopsticks too quickly, allowing some to secure both chopsticks before others could grasp their left one.

Solution: To address this, a deliberate delay (`usleep(500000)`) was introduced after each philosopher picks up their left chopstick. This adjustment ensures all philosophers hold their left chopstick before trying to grab the right one, thereby establishing a circular wait.

```
sem_wait(&p->left_chop->sem);
usleep(500000);  // CRITICAL: Forces deadlock
sem_wait(&p->right_chop->sem);
```

7.2 Challenge 2: Display Corruption and Text Overlap

Problem: The terminal display was plagued by overlapping text, causing legend lines to appear amid the philosopher state updates (e.g., "Time: 9 secKNING").

Solution:

- The legend was moved inside the `update_display()` function to guarantee it prints with each refresh.
- Cursor positioning (`\033[H`) was utilized instead of completely clearing the screen to reduce flickering.
- Appropriate newline characters were inserted throughout to improve formatting.

7.3 Challenge 3: Compilation Warnings

Problem: Several functions in `stats.c` produced warnings about unused parameters.

Solution: `(void)` casts were added to clearly signal that these parameters are intentionally not utilized.

```
void stats_record_wait_time(SimulationStats *stats, int phil_id, long wait_us) {
    (void)stats; (void)phil_id; (void)wait_us;
}
```

7.4 Challenge 4: Thread Cleanup on Early Exit

Problem: Pressing Ctrl+C terminated the main thread while the philosopher threads continued to run, resulting in resource leaks.

Solution: A signal handler was put in place to set `simulation_running = false`, enabling the threads to exit their loops gracefully. This approach guarantees that all threads are joined before the program concludes.

```
void signal_handler(int sig) {
    (void)sig;
    simulation_running = false;
}
```

7.5 Challenge 5: Package Manager Lock

Problem: During setup, an issue arose with `sudo apt install` that stalled due to an automatic system update lock.

Solution: The halted process was terminated, and the lock files were cleared, enabling the successful installation of `build-essential`.

```
sudo kill -9 5149
sudo rm /var/lib/dpkg/lock-frontend
sudo apt install -y build-essential
```

8. Conclusion

8.1 Project Summary

This project successfully developed a comprehensive Dining Philosophers simulation in C, achieving all the objectives set out at the beginning. We created a fully functional simulator that includes two distinct versions: one susceptible to deadlocks and freezing, and a deadlock-free variant that employs resource hierarchy to ensure continuous progress. The program features real-time terminal visualization with color-coded states, thorough CSV logging of state transitions, and collection of performance metrics. Testing with 1 to 10 philosophers demonstrated that the deadlock-free mode consistently completed tasks successfully, while the deadlock mode effectively showcased the classic circular wait condition.

8.2 Key Learnings

The project provided valuable insights into several fundamental operating system concepts. We learned that all four Coffman conditions must be true at the same time for a deadlock to happen; breaking even one of these conditions can prevent it. The use of resource hierarchy successfully eliminates circular waiting without the need for complex recovery processes. Our experience with POSIX threads and semaphores highlighted the critical importance of proper initialization and cleanup. We also examined the significance of the `volatile` keyword for flags shared between threads, as well as the role of signal handling in multi-threaded programs to ensure graceful shutdowns.

8.3 Real-World Applications

The principles demonstrated in this project have direct applications in real-world systems. For instance, database management systems often face similar challenges with transaction locking mechanisms that can lead to deadlocks. File systems rely on synchronization primitives to manage concurrent access from multiple processes. Memory management units employ locking protocols for shared memory allocation, while device drivers managing hardware resources like printers or disk drives must implement strategies to avoid deadlock. A solid understanding of these issues provides foundational knowledge relevant to concurrent systems, including embedded real-time operating systems (RTOS) and cloud infrastructures.

8.4 Future Improvements

Several enhancements could take this project to the next level. For example, adding a FIFO queue for hungry philosophers could help prevent starvation. Developing a graphical user interface using GTK or Qt would enhance visualization. Enabling dynamic configuration at runtime would allow for adjustments to thinking and eating durations without necessitating a restart. Implementing a watchdog thread could enable automatic detection of deadlock conditions and provide valuable diagnostic output. Additionally, creating a networked version could extend the project into distributed systems, demonstrating how these challenges evolve across network boundaries.

8.5 Final Remarks

The Dining Philosophers problem serves as a powerful illustration of the concurrency challenges found in operating systems. This project brought abstract textbook concepts into tangible code that not only demonstrates the problem but also its solution. Observing the terminal freezes in deadlock mode highlights the impact of resource acquisition strategies, while the smooth operation of the safe mode reaffirms that thoughtful algorithm design ensures continuous progress. Overall, this project adds to practical skills in multi-threaded programming, synchronization primitives, and defensive coding that are essential for systems programming roles in the industry

philosopher_id	meals_eaten	
0	22	
1	24	
2	25	
3	25	
4	25	
5	25	
6	26	
7	27	
8	28	
9	23	
# total_meals	250	
# avg_wait_us	5289571.9	
# max_wait_us	7458681	

Figure 12: Final Program Output Overview

9. Appendix

1. **What are the four Coffman conditions for deadlock?**
   Mutual exclusion, hold and wait, no preemption, and circular wait. Our safe version breaks the circular wait by enforcing a global order for picking up chopsticks.

2. **How does your resource hierarchy solution prevent deadlock?**
   Each philosopher always picks up the chopstick with the lower ID first, then the higher ID. This eliminates circular wait because the last philosopher picks the first chopstick, breaking the cycle.

3. **Why did you choose semaphores instead of mutexes?**
   Semaphores are the classic solution for the Dining Philosophers problem and can be used as binary locks (mutex). They also allow easy extension to counting semaphores if we had multiple resources.

4. **Can starvation occur in your safe implementation?**
   Starvation is unlikely due to random think/eat times, but in theory, it could happen. A fairness mechanism (e.g., a queue for hungry philosophers) would eliminate it.

5. **What happens when you press Ctrl+C during the simulation?**
   A signal handler sets simulation_running = false, causing all philosopher threads to exit their loops gracefully. The main thread then joins them and cleans up resources.

6. **Why did your deadlock version initially not deadlock every time?**
   Without a forced delay, philosophers could acquire both chopsticks before others grabbed their left one. Adding a 0.5-second delay after picking left ensures everyone holds left simultaneously, causing a circular wait.

7. **How do you log state changes, and why is CSV format useful?**
   We write each state change with a timestamp to a CSV file. CSV allows easy import into Excel/Python for plotting meals over time and analyzing waiting times.

8. **What is the purpose of the volatile keyword for simulation_running?**
   It prevents the compiler from optimizing away the variable's checks inside threads, ensuring that when the signal handler modifies it, all threads see the updated value immediately.

9. **How do you ensure that two philosophers do not use the same chopstick simultaneously?**
   Each chopstick is protected by a binary semaphore initialised to 1. A philosopher must call sem_wait() before using it, which blocks if the chopstick is already taken.

10. **Could your safe mode work with an odd number of philosophers? With even?**
    Yes, the resource hierarchy works for any number ≥ 2. It breaks circular wait regardless of parity; even numbers still deadlock in the naive version, but our safe version prevents it.

11. **How do you measure waiting time for each philosopher?**
    We record the timestamp when a philosopher enters HUNGRY and subtract it from the timestamp when they acquire both chopsticks. The difference is accumulated in total_wait_time.

12. **What real-world systems use similar deadlock avoidance techniques?**
    Database locking (e.g., two-phase locking), operating system file system locks, and network resource allocation often use ordered acquisition to prevent deadlocks.

- End of Lab Report -